

ECOM F342 COMPUTER ARCHITECTURE ASSIGNMENT

Implement a 6-stage pipelined processor in Verilog. This processor supports **load (lw), store (sw), jump (j), add (add), and (and), and add immediate (addi) instructions only**. The processor should implement forwarding to resolve data hazards. The processor has Reset, CLK as inputs and no outputs. The processor has instruction fetch unit, decode, reg read (with 32 32-bit registers), execution, memory and writeback units. The processor also contains five pipelined registers IF/ID, ID/RR, RR/EX, EX/MEM and MEM/WB. When reset is activated the PC, IF/ID, ID/RR, RR/EX, EX/MEM and MEM/WB registers are initialized to 0, the instruction memory and register file get loaded by predefined values. When the instruction unit starts fetching the first instruction the pipelined registers contain unknown values. When the second instruction is being fetched in the IF unit, the IF/ID register will hold the instruction code for the first instruction. When the third instruction is being fetched by the IF unit, the IF/ID register contains the instruction code of the second instruction, the ID/RR register contains information related to the first instruction and so on. (Assume a 32-bit PC. Also Assume Address and Data size as 32-bit).

The instruction and its 32-bit instruction format are shown below:

lw destinationReg, offset [sourceReg] (Sign extends data specified in instruction field (15:0) to 32-bits, add it with register specified by register number in rs field and store the data corresponding to the memory location defined by the calculated address in the rt register. Opcode for lw is 100011).

op	rs	rt	offset
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	16-bits (15-0)

sw sourceReg, offset [destinationReg] (Sign extends data specified in instruction field (15:0) to 32-bits, add it with register specified by register number in rs field. Opcode for sw is 101011).

op	rs	rt	offset
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	16-bits (15-0)

j target (Shift left by 2 the data specified in offset field (25:0) to 28-bits, and append the first 4 bits of PC+4. Opcode for j is 000010).

op	offset
6 bits (31-26)	26-bits (25-0)

add sourceReg 1, sourceReg 2, destinationReg (Perform add operation on the registers specified by registers specified by register numbers in rs and rt fields and save the result in the register specified by register number in rd field. Opcode for add is 000000 and function is 100000).

op	rs	rt	rd	shamt	funct
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	5-bits (15-11)	5-bits (10-6)	6-bits (5-0)

and sourceReg 1, sourceReg 2, destinationReg (Perform and operation on the registers specified by registers specified by register numbers in rs and rt fields and save the result in the register specified by register number in rd field. Opcode for and is 000000 and function is 100100).

op	rs	rt	rd	shamt	funct
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	5-bits (15-11)	5-bits (10-6)	6-bits (5-0)

addi destinationReg, sourceReg, immediate (Sign extends data specified in instruction field (15:0) to 32-bits, add it with register specified by register number in rs field. And store the result in rt. Opcode for addi is 001000).

op	rs	rt	offset
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	16-bits (15-0)

Assume the register file contains 32 registers (R0-R31) each register can hold 32-bit data. On reset, PC and all register file registers should get initialized to 0. Ensure r0 is always zero. Each location in DMEM has 8-bit data. So, to store a 32-bit value, you need 4 locations in the DMEM, stored in big-endian format. Let DMEM[10] have the value 8'd96 on reset. Also ensure that on reset, the instruction memory gets initialized with the following instructions, starting at address 0:

```

addi R1, R2, #10
addi R4, R3, #5
add R7, R6, R1
and R8, R7, R4
sw R8, 4(R3)
lw R9, 4(R3)
add R10, R9, R1
j T
add R11, R2, R3

```

T:

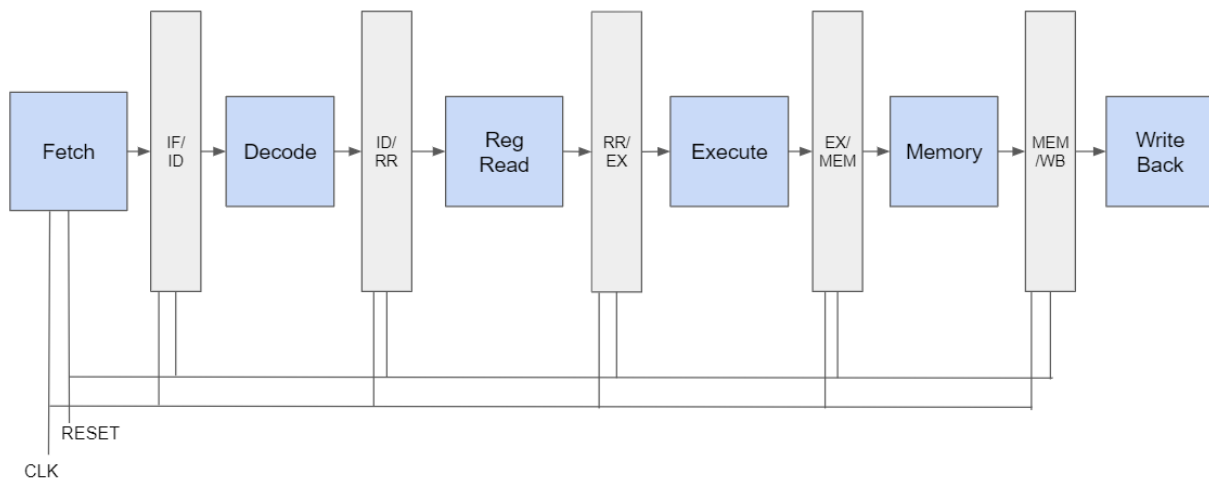
```

and R12, R10, R8

```

The above code should work correctly on the processor implementation. Ensure that data hazards, if any, are also handled.

A partial block level representation of the 6-stage pipelined processor is shown below. **Please note that for registerfile implementation, write should be on the positive edge and read should be on the negative edge of the clock.** Write operation depends on the control signal.



As part of the assignment three files should be submitted in a zipped folder.

1. PDF version of this Document with all the Questions below answered with file name as **IDNO_NAME.pdf**.
2. Design Verilog Files for all the Sub-modules (instruction fetch, Register file, forwarding unit).
3. Design Verilog file for the main processor.

The name of the zipped folder should be in the format IDNO_NAME.zip

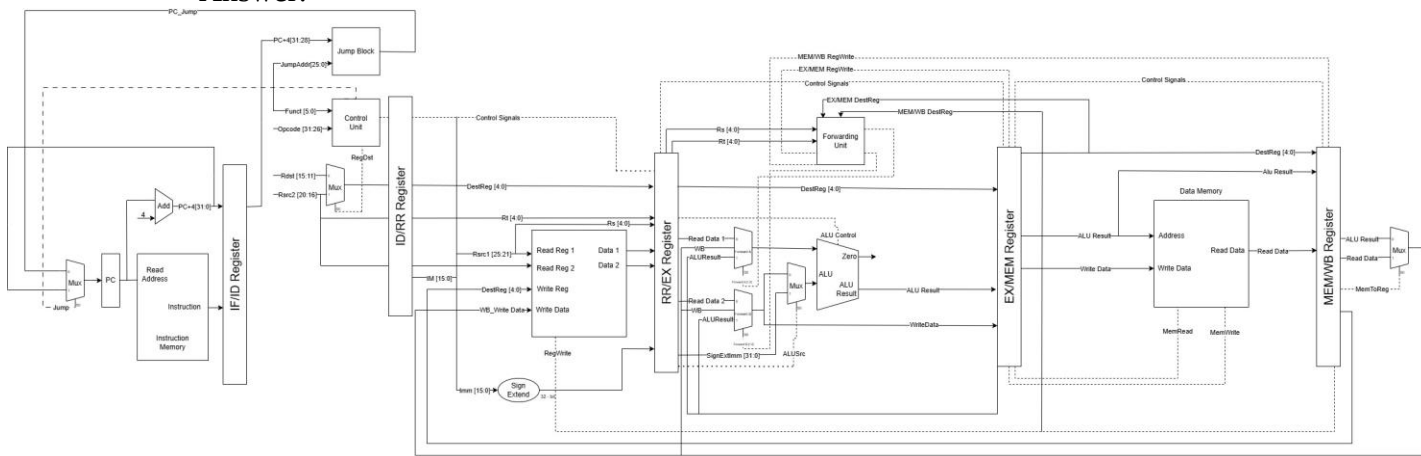
NAME: Manan Agarwal

ID No: 2022B5A30129G

Questions Related to Assignment

1. Draw the complete Datapath and show control signals of the 6-stage pipelined processor. A sample Datapath for 5-stage pipelined MIPS processor has been discussed in class. A ppt named Assignmenthelp.ppt contains this 5-stage processor and is uploaded in CMS. You can modify this according to your specification.

Answer:



2. List the control signals used and also the values of control signals for different instructions in a tabular format as follows:

Answer:

Instructions	Control Signals					
	RegWrite	MemWrite	ALUSrc	ALU_ctrl	MemToReg	PCSrc
lw	1	0	1	0010	1	0
sw	0	1	1	0010	0	0
j	0	0	x	xxxx	0	1
add	1	0	0	0010	0	0
and	1	0	0	0000	0	0
addi	1	0	1	0010	0	0

3. In a program, there are 25% load instructions, 1/x of which are immediately followed by an instruction that uses a result, requiring a stall. 10% are stores. 50% are R-type. 10% are branch, 1/y of which are taken. 5% are jumps. What is the average CPI of this program? If the number of instructions are 10^9 , and the clock cycle is 100 ps, how much time does a MIPS single cycle pipelined processor take to execute all instructions? Assume the processor always predicts branch not-taken.

Where x, y, z are related to last 3 digits of your ID No.

If ID number: 20XXXXXXABCG, then $x = (A \% 8) + 1$, $y = ((B + 2) \% 8) + 1$, and $z = ((C + 3) \% 8) + 1$.

Answer: A = 1, B = 2, C = 9

$x = 2$, $y = 5$, $z = 5$

Average CPI = 1.185 (assuming Branch Penalty = 3)

Execution Time = 0.1185 seconds

4. Implement the Instruction Fetch block. Copy the image of Verilog code of the Instruction fetch block here

Answer:

```
module Instruction_Fetch (
    input clk, reset, PCSrc, Stall,
    input [31:0] Target_Address,
    output [31:0] Instruction
);
    reg [31:0] PC;
    Instruction_Memory IM1 (.PC(PC), .reset(reset), .Instruction(Instruction));

    always @(posedge clk or negedge reset) begin
        if (reset == 0) PC <= 0;
        else if (PCSrc) PC <= Target_Address;
        else if (Stall) PC <= PC;
        else PC <= PC + 4;
    end
endmodule

module IF_ID_Reg(
    input [31:0] Instruction,
    input clk, reset, Stall,
    output reg [31:0] Instruction_Code
);
    always @(posedge clk or negedge reset) begin
        if (reset == 0) Instruction_Code <= 0;
        else if (Stall) Instruction_Code <= Instruction_Code;
        else Instruction_Code <= Instruction;
    end
endmodule
```

5. Implement the Instruction Decode block. Copy the image of Verilog code of the Instruction decode block here

Answer:

```
module Instruction_Decode(
    input [31:0] Instruction_Code,
    input reset, Stall,
    output reg [4:0] RS, RT, RD,
    output reg RegWrite, ALUSrc, MemWrite, MemToReg, PCSrc, DestSrc,
    output reg [3:0] ALU_control_lines,
    output reg [25:0] Jump_Offset,
    output reg [15:0] Const
);
    always @(*) begin
        {RS, RT, RD, RegWrite, ALUSrc, MemWrite, MemToReg, PCSrc, DestSrc, ALU_control_lines, Jump_Offset, Const} = 0;
        if (!Stall) begin
            case (Instruction_Code[31:26])
                6'b001000: begin // ADDI
                    RS = Instruction_Code[25:21]; RT = Instruction_Code[20:16];
                    Const = Instruction_Code[15:0]; RegWrite = 1; ALUSrc = 1; DestSrc = 1; ALU_control_lines = 4'b0010;
                end
                6'b100011: begin // LW
                    RS = Instruction_Code[25:21]; RT = Instruction_Code[20:16];
                    Const = Instruction_Code[15:0]; RegWrite = 1; ALUSrc = 1; MemToReg = 1; DestSrc = 1; ALU_control_lines = 4'b0010;
                end
                6'b101011: begin // SW
                    RS = Instruction_Code[25:21]; RT = Instruction_Code[20:16];
                    Const = Instruction_Code[15:0]; MemWrite = 1; ALUSrc = 1; ALU_control_lines = 4'b0010;
                end
                6'b000000: begin // R-Type
                    RS = Instruction_Code[25:21]; RT = Instruction_Code[20:16]; RD = Instruction_Code[15:11];
                    RegWrite = 1; case(Instruction_Code[5:0]) 6'b100000: ALU_control_lines = 4'b0010; 6'b100100: ALU_control_lines = 4'b0000; endcase
                end
                6'b000010: begin // Jump
                    PCSrc = 1; Jump_Offset = Instruction_Code[25:0];
                end
            endcase
        end
    end
endmodule
```

6. Determine the condition that can be used to detect data hazard?

Answer:

(RegWrite_ID_RF=1) AND (Write_Reg_ID_RF!=0) AND
(Write_Reg_ID_RF=RS_IF_ID) OR (Write_Reg_ID_RF=RT_IF_ID)

Implement the Register File and copy the image of Verilog code of Register file unit here.

Answer:

```
module Register_File (
    input [4:0] RS, RT, Write_Reg_Num,
    input [31:0] Write_Data,
    input RegWrite, clk, reset,
    output reg [31:0] RS_Data, RT_Data
);
    reg [31:0] RegMemory [31:0];
    integer i;

    always @(negedge clk) begin
        RS_Data <= RegMemory[RS];
        RT_Data <= RegMemory[RT];
    end
    always @(posedge clk) if (RegWrite && Write_Reg_Num != 0) RegMemory[Write_Reg_Num] <= Write_Data;
    always @(negedge reset) begin
        for (i=0; i<32; i=i+1) RegMemory[i] <= 0;
    end
endmodule
```

7. Implement the forwarding unit and copy the image of Verilog code of forwarding unit here.

```
module Forwarding_Unit (
    // Source operands from IF_ID register (next instruction being decoded)
    input [4:0] RS_IF_ID, RT_IF_ID,
    // Destination register resolved in ID_RF stage (Rd or Rt per DestSrc)
    input [4:0] Write_Reg_ID_RF,
    // Destination and write-enable from EX/MEM stage (for MEM-stage forwarding)
    input [4:0] Write_Reg_EX_MEM,
    input RegWrite_ID_RF, RegWrite_EX_MEM,
    output reg [1:0] A_Forward, B_Forward
);

always @(*) begin
    A_Forward = 2'b00;
    B_Forward = 2'b00;

    // EX forwarding: destination resolved in ID_RF vs operands in IF_ID
    // 2'b10 => forward from ID_RF result (one stage ahead)
    if (RegWrite_ID_RF && Write_Reg_ID_RF != 0 && Write_Reg_ID_RF == RS_IF_ID)
        A_Forward = 2'b10;
    else if (RegWrite_EX_MEM && Write_Reg_EX_MEM != 0 && Write_Reg_EX_MEM == RS_IF_ID)
        A_Forward = 2'b01;

    if (RegWrite_ID_RF && Write_Reg_ID_RF != 0 && Write_Reg_ID_RF == RT_IF_ID)
        B_Forward = 2'b10;
    else if (RegWrite_EX_MEM && Write_Reg_EX_MEM != 0 && Write_Reg_EX_MEM == RT_IF_ID)
        B_Forward = 2'b01;
end
endmodule
```

Answer:

8. Implement a complete processor in Verilog (using all the Datapath blocks). Copy the image of Verilog code of the processor here. (Use comments to describe your Verilog implementation)

```
module Processor (
    input clk,
    input reset
);

// --- Wire Definitions for Stage Interconnections ---
// IF Stage
wire [31:0] IF_Instruction, PC_plus_4, Target_Address;

// ID Stage
wire [31:0] ID_Instruction;
wire [4:0] ID_RS, ID_RT, ID_RD;
wire [3:0] ID_ALU_Control;
wire [25:0] ID_Jump_Offset;
wire [15:0] ID_Const;
wire ID_RegWrite, ID_ALUsrc, ID_MemWrite, ID_MemToReg, ID_PCsrc, ID_DestSrc;

// RR Stage
wire [31:0] RR_RS_Data, RR_RT_Data, RR_SignExtended;
wire [31:0] RR_B_Mux_Out;
wire [4:0] RR_RS, RR_RT, RR_RD, RR_Write_Reg_Out;
wire [3:0] RR_ALU_Control;
wire RR_RegWrite, RR_MemWrite, RR_MemToReg, RR_ALUsrc, RR_DestSrc;

// EX Stage
wire [31:0] EX_A, EX_B, EX_RT_Data, EX_ALU_Result;
wire [31:0] EX_Forward_A_Mux, EX_Forward_B_Mux;
wire [4:0] EX_Write_Reg, EX_RS, EX_RT;
wire [3:0] EX_ALU_Control;
wire EX_RegWrite, EX_MemWrite, EX_MemToReg, EX_Zero;
wire [1:0] ForwardA, ForwardB;

// MEM Stage
wire [31:0] MEM_ALU_Result, MEM_RT_Data, MEM_Data_Out;
wire [4:0] MEM_Write_Reg;
wire MEM_RegWrite, MEM_MemWrite, MEM_MemToReg;

// WB Stage
wire [31:0] WB_Mem_Data, WB_ALU_Result, WB_Final_Write_Data;
wire [4:0] WB_Write_Reg;
wire WB_RegWrite, WB_MemToReg;

// Hazard & Flush
wire Stall, Flush;
```

Answer:

```

wire [15:0] ID_Const_Out; // To catch the output from the ID_RR register
wire [31:0] SignExtendedImmediate;

// Source operands of the instruction currently in IF/ID
// (the instruction one stage behind ID_RF, i.e. the next instruction to be decoded)
wire [4:0] RS_IF_ID = ID_Instruction[25:21];
wire [4:0] RT_IF_ID = ID_Instruction[20:16];

// -----
// 1. FETCH STAGE (IF)
// -----
Instruction_Fetch IF_Unit (
    .clk(clk), .reset(reset), .Stall(Stall), .PCSrc(ID_PCSrc),
    .Target_Address(Target_Address), .Instruction(IF_Instruction)
);

IF_ID_Reg IF_ID (
    .clk(clk), .reset(reset && !Flush), .Stall(Stall),
    .Instruction(IF_Instruction), .Instruction_Code(ID_Instruction)
);

// -----
// 2. DECODE STAGE (ID)
// -----
Instruction_Decode ID_Unit (
    .Instruction_Code(ID_Instruction), .Stall(Stall), .reset(reset), .RS(ID_RS), .RT(ID_RT),
    .RD(ID_RD), .RegWrite(ID_RegWrite), .ALU_control_lines(ID_ALU_Control),
    .ALUSrc(ID_ALUSrc), .MemWrite(ID_MemWrite), .MemToReg(ID_MemToReg),
    .PCSrc(ID_PCSrc), .Jump_Offset(ID_Jump_Offset), .DestSrc(ID_DestSrc), .Const(ID_Const)
);

// Jump logic: Target = {PC[31:28], Offset << 2}
assign Target_Address = {4'b0000, ID_Jump_Offset, 2'b00};

// --- Control Signal Bubble Mux ---
wire Stall_RegWrite = (Stall) ? 1'b0 : ID_RegWrite;
wire Stall_MemWrite = (Stall) ? 1'b0 : ID_MemWrite;

```

```

ID_RF_Reg ID_RF (
    .clk(clk), .reset(reset), .RegWrite_in(Stall_RegWrite), .MemWrite_in(Stall_MemWrite),
    .MemToReg_in(ID_MemToReg), .ALUSrc_in(ID_ALUSrc), .ALU_control_in(ID_ALU_Control),
    .RS_in(ID_RS), .RT_in(ID_RT), .RD_in(ID_RD), .Const_in(ID_Const), .DestSrc_in(ID_DestSrc),
    .RegWrite_out(RR_RegWrite), .MemWrite_out(RR_MemWrite), .MemToReg_out(RR_MemToReg),
    .ALUSrc_out(RR_ALUSrc), .ALU_control_out(RR_ALU_Control), .RS_out(RR_RS),
    .RT_out(RR_RT), .RD_out(RR_RD), .DestSrc_out(RR_DestSrc), .Const_out(ID_Const_Out)
);

// Destination register resolved in ID_RF stage: Rd (R-type) or Rt (I-type)
// RR_Write_Reg_Out = (RR_DestSrc) ? RR_RT : RR_RD
assign RR_Write_Reg_Out = (RR_DestSrc) ? RR_RT : RR_RD;

// -----
// 3. REGISTER READ STAGE (RR)
// -----
Register_File RF (
    .clk(clk), .reset(reset), .RS(RR_RS), .RT(RR_RT),
    .Write_Reg_Num(WB_Write_Reg), .Write_Data(WB_Final_Write_Data),
    .RS_Data(RR_RS_Data), .RT_Data(RR_RT_Data), .RegWrite(WB_RegWrite)
);

assign RR_SignExtended = {{16{ID_Const_Out[15]}}, ID_Const_Out};
assign RR_B_Mux_Out = (RR_ALUSrc) ? RR_SignExtended : RR_RT_Data;

RF_EX_Reg RF_EX (
    .clk(clk), .reset(reset), .A_in(RR_RS_Data), .B_in(RR_B_Mux_Out),
    .RT_Data_in(RR_RT_Data), .Write_Reg_in(RR_Write_Reg_Out), .RS_in(RR_RS), .RT_in(RR_RT),
    .RegWrite_in(RR_RegWrite), .MemWrite_in(RR_MemWrite), .MemToReg_in(RR_MemToReg), .ALU_control_in(RR_ALU_Control),
    .RS_out(EX_RS), .RT_out(EX_RT), .A_out(EX_A), .B_out(EX_B), .RT_Data_out(EX_RT_Data),
    .Write_Reg_out(EX_Write_Reg), .RegWrite_out(EX_RegWrite), .MemWrite_out(EX_MemWrite),
    .MemToReg_out(EX_MemToReg), .ALU_control_out(EX_ALU_Control)
);

```

```

Forwarding_Unit FU (
    .RS_IF_ID(RS_IF_ID),
    .RT_IF_ID(RT_IF_ID),
    .Write_Reg_ID_RF(RR_Write_Reg_Out),
    .RegWrite_ID_RF(RR_RegWrite),
    .Write_Reg_EX_MEM(MEM_Write_Reg),
    .RegWrite_EX_MEM(MEM_RegWrite),
    .A_Forward(ForwardA),
    .B_Forward(ForwardB)
);

// 3x1 MUXes for Forwarding
assign EX_Forward_A_Mux = (ForwardA == 2'b10) ? MEM_ALU_Result :
    (ForwardA == 2'b01) ? WB_Final_Write_Data : EX_A;
assign EX_Forward_B_Mux = (ForwardB == 2'b10) ? MEM_ALU_Result :
    (ForwardB == 2'b01) ? WB_Final_Write_Data : EX_B;

ALU_Main_ALU (
    .A(EX_Forward_A_Mux), .B(EX_Forward_B_Mux),
    .ALU_control_lines(EX_ALU_Control), .zero(EX_Zero), .ALU_result(EX_ALU_Result)
);

EX_MEM_Reg_EX_MEM (
    .clk(clk), .reset(reset), .ALU_result_in(EX_ALU_Result), .RT_Data_in(EX_RT_Data),
    .Write_Reg_in(EX_Write_Reg), .MemWrite_in(EX_MemWrite), .RegWrite_in(EX_RegWrite),
    .MemToReg_in(EX_MemToReg), .ALU_result_out(MEM_ALU_Result), .RT_Data_out(MEM_RT_Data),
    .Write_Reg_out(MEM_Write_Reg), .MemWrite_out(MEM_MemWrite), .RegWrite_out(MEM_RegWrite),
    .MemToReg_out(MEM_MemToReg)
);

// -----
// 5. MEMORY STAGE (MEM)
// -----
Data_Memory_DMEM (
    .clk(clk), .reset(reset), .MemWrite(MEM_MemWrite),
    .MemAddress(MEM_ALU_Result), .Write_Data(MEM_RT_Data), .Mem_Data(MEM_Data_Out)
);

```

```

MEM_WB_Reg_MEM_WB (
    .clk(clk), .reset(reset), .Mem_Data_in(MEM_Data_Out), .ALU_result_in(MEM_ALU_Result),
    .Write_Reg_in(MEM_Write_Reg), .RegWrite_in(MEM_RegWrite), .MemToReg_in(MEM_MemToReg),
    .Mem_Data_out(WB_Mem_Data), .ALU_result_out(WB_ALU_Result), .Write_Reg_out(WB_Write_Reg),
    .RegWrite_out(WB_RegWrite), .MemToReg_out(WB_MemToReg)
);

// -----
// 6. WRITE BACK STAGE (WB)
// -----
assign WB_Final_Write_Data = (WB_MemToReg) ? WB_Mem_Data : WB_ALU_Result;

// -----
// HAZARD & FLUSH UNIT
// -----
Stall_Unit_Hazard_Control (
    .RS_ID_RF(RR_RS), .RT_ID_RF(RR_RT), .Write_Reg_RF_EX(EX_Write_Reg),
    .MemToReg_RF_EX(EX_MemToReg), .Stall(Stall)
);

endmodule

```

- Test the processor design by generating the appropriate clock and reset. Copy the image of your testbench code here.

```

module Processor_tb;
    reg clk;
    reg reset;

    Processor DUT(
        .clk(clk),
        .reset(reset)
    );

    initial begin
        clk = 1; // Initialize clock to 0
        reset = 1;
        #2 reset = 0;
        #2 reset = 1;

        #161;
        $finish;
    end

    // Clock generation 10ns time period
    always #5 clk = ~clk;
endmodule

```

Answer:

10. Verify if the register file is getting updated according to the set of instructions (mentioned earlier).

Copy verified **Register file** waveform here (show only the Registers that get updated, CLK, and RESET):



11. What are the total number of cycles needed to issue the program given above on the pipelined MIPS Processor? What is the CPI of the program?

Answer: 16 cycles are needed in total to execute the above program, which makes the CPI 1.6 since there are 10 instructions, although one of those gets skipped due to jump. 9 instructions through 6 stages = 14 clock cycles, and 1 stall and 1 flush for load word and jump.

12. Make a diagram showing the clock by clock execution of each instruction, indicating stalling, forwarding etc wherever necessary.

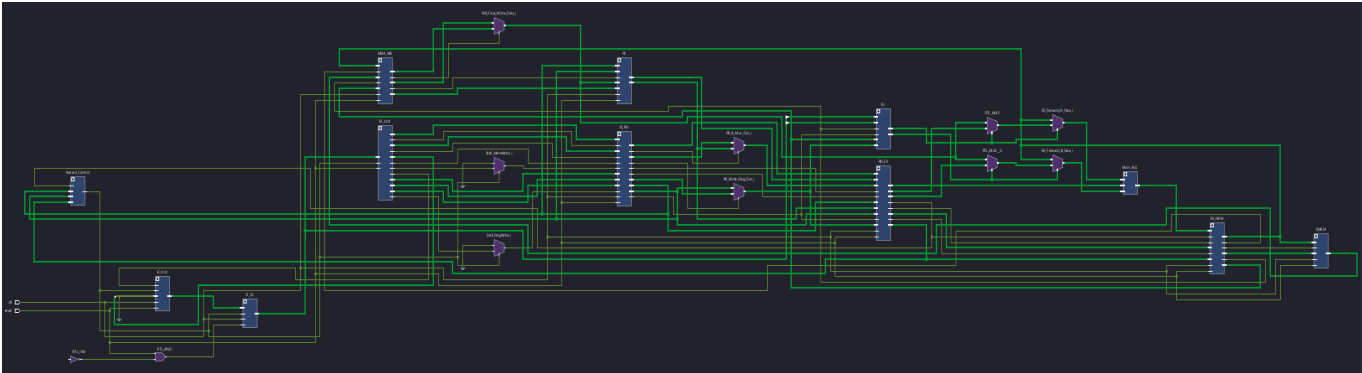
Answer



14. Your design synthesisable? Which target FPGA was used for synthesis?

Answer:

Yes the design is synthesisable. Xilinx Kintex-7 FPGA (xc7k70tfbv676-1) was used for synthesis



15. Provide the synthesis report in tabular form (resources consumed)?

Answer:

Site Type	Used	Available	Utilization %
Slice LUTs	0	41,000	0.00
Slice Registers	0	82,000	0.00
F7 Muxes	0	20,500	0.00
F8 Muxes	0	10,250	0.00

Site Type	Used	Available	Utilization %
Block RAM Tile	0	135	0.00
DSPs	0	240	0.00
Bonded IOB	0	300	0.00

Component Type	Element Description	Count
Adders	2 Input, 32-bit Adder	1
Registers	32-bit Registers	10
	16-bit Registers	1
	8-bit Registers	148
	5-bit Registers	12
	4-bit Registers	2
	1-bit Registers	13
Muxes	2-input, 32-bit Muxes	3
	3-input, 32-bit Muxes	3
	2-input, 8-bit Muxes	126

Component Type	Element Description	Count
	4-input, 8-bit Muxes	64
	2-input, 1-bit Muxes	160
	5-input, 1-bit Muxes	90
	Others (Various widths/inputs)	20

Unrelated Questions

What were the problems you faced during the implementation of the processor?

Answer: I faced problems in implementing the forwarding and stall logic in this specific question as there are 6 stages in this processor, and I couldn't decide whether to use ID/RR register content or RR/EX register content for hazard detection. Also it was difficult to keep a track of everything (inputs, outputs, control signals) while writing the modules, and then finally debugging via waveforms of the register content, control signals and Instructions.

Did you implement the processor on your own? If you took help from someone whose help did you take? Which part of the design did you take help for?

Answer: Yes I implemented the processor and submodules of the processor on my own, however I did take help from my friends in discussing the logic for forwarding and stalling as this hadn't been implemented by us in the previous labs.

Honor Code Declaration by student:

- My answers to the above questions are my own work.
- I have not shared the codes/answers written by me with any other students. (I might have helped clear doubts of other students).
- I have not copied other's code/answers to improve my results. (I might have got some doubts cleared from other students).

Name: Manan Agarwal
ID No.: 2022B5A30129G

Date: 14th April 2026